

Hot–Cold Tiering of HBM and High Bandwidth Flash for Agentic LLM Serving

Jongjin Baek, Won Ji, Seungjae Yoo and Joo-Young Kim

Abstract—Large language model (LLM) serving is increasingly agentic, with multi-turn sessions that idle between actions yet must retain their full context. Limited GPU memory capacity forces inactive KV states to be evicted, so resuming a session incurs either costly recomputation or slow interconnect transfers. To address this, high bandwidth flash (HBF)—an on-package 3D-NAND memory offering orders-of-magnitude greater capacity than high bandwidth memory (HBM) at comparable read bandwidth—has emerged as a strong candidate. However, its high read energy and limited write endurance make it impractical to serve *all* KV traffic. Fortunately, our analysis shows that agentic KV states exhibit distinct access patterns: a small hot set is read for every decoding step, while a large cold pool is read only when a paused session resumes. Exploiting this, we place the hot set in HBM and the cold pool in HBF, forming a hot–cold KV hierarchy within the GPU memory tier. On agentic workloads with Qwen3-Coder-30B-A3B, our design delivers 14 ms time-between-tokens (TBT) and adds only ≈ 0.1 ms of resume latency on top of prefill, while hosting $24\times$ more concurrent sessions per GPU. By confining HBM to the hot set, our design also cuts read power by 7.6 kW per 8-GPU node relative to serving all KV from flash—establishing HBF as a cold-tier complement to HBM rather than its replacement.

Index Terms—LLM inference, KV cache, high bandwidth flash, memory hierarchy, agentic serving, hot–cold tiering.

I. INTRODUCTION

Large Language Model (LLM) inference is becoming agentic. A single-query session now turns into a multi-turn loop of decoding, tool calls, code execution, and user input [1], [2]. Each turn appends the output of the model, the tool results, and the next user message. The key-value (KV) cache then accumulates across the entire session instead of resetting per query (Fig. 1). The KV cache is read at every active decoding step but remains idle and pauses during long waits for a tool or user input until it resumes [3], [4]. Consequently, the KV cache is split into two contrasting groups. The first is a small, frequently accessed set (KV of active sessions), and the second is a large, infrequently accessed pool (KV of paused sessions), which is read only upon resumption. Meeting tight time-between-tokens (TBT) service-level objectives (SLOs) requires keeping the active KV cache resident in HBM for fast access. The footprint of this actively read state which grows with batch size stays small for two reasons. First, HBM capacity is finite. Second, under tight TBT SLOs, increasing the decode batch lowers per-request throughput [2]. The paused state behaves differently. Because agentic workflows feature long periods of inactivity [4], the aggregate KV cache of paused sessions grows continuously over time, overflowing HBM well before a node hits its production session target [5]. To support more concurrent sessions, systems risk violating

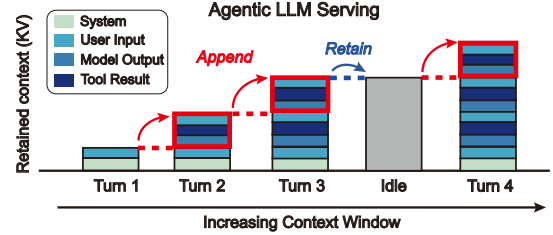


Fig. 1. Characteristics of agentic AI

their SLOs by offloading KV states to slower memory or discarding them, leading to expensive context recomputation on resume. This limitation is fundamentally driven by the bimodality of agentic KV cache. Paused sessions require high capacity, while both active and paused sessions require high bandwidth, and no single memory technology serves both well. We address this bandwidth–capacity mismatch by integrating high bandwidth flash (HBF) into the GPU memory hierarchy.

HBF is an emerging memory technology that stacks 3D-NAND behind an HBM-style interface, offering greater capacity than HBM at a lower cost per bit [6]. Compared to HBM, however, HBF suffers from higher latency, significantly higher per-bit read energy [7], and lower write endurance [8], which fundamentally limits write rates. Mitigating these limitations requires integrating HBF into the memory hierarchy such that its workloads naturally mask its energy and endurance overheads. Our contributions are as follows: (1) identifying that KV access in agentic LLM is bimodal with a hot set read every step and a cold pool read only on resume, (2) data movement and placement policy on a heterogeneous memory architecture exploiting this bimodality, keeping the hot set in HBM and the cold pool in HBF, (3) trace-based simulation demonstrating that our design expands session capacity and ensures low TBT, while mitigating high HBF read energy and preserving flash endurance within warranty.

II. BACKGROUND AND MOTIVATION

Large Language Models (LLMs) generate text autoregressively in two phases. *Prefill* phase processes the prompt in parallel to compute key-value (KV) tensors, and *decoding* phase generates tokens sequentially by attending to past KV data. Systems maintain a per-session *KV cache* to avoid recomputation [9]. Because this cache grows linearly with context length and concurrency, decoding is memory-bandwidth bound. Modern frameworks like vLLM [9] allocate the cache in fixed-size blocks to eliminate capacity waste. Since autoregressive decoding only appends to the cache, a KV entry is never modified once written and is therefore immutable. We assume this setting throughout, excluding policies that rewrite KV in place such as in-session compression.

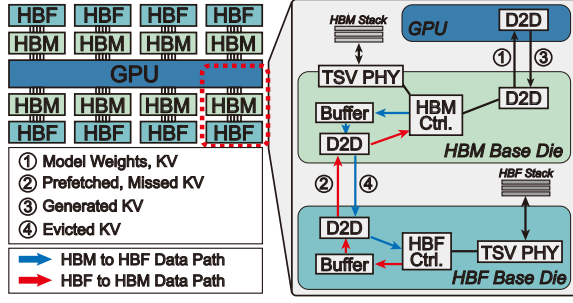


Fig. 2. Tiered HBM+HBF architecture. *Left*: each of the memory sites of GPU co-packages an HBF stack on an HBM stack. *Right*: Zoomed view showing the GPU connected to HBM via a die-to-die (D2D) link.

Agentic AI Serving wraps a model in a multi-step loop, interleaving model calls with tool execution or user inputs that feed back into a growing context [10]. This creates two distinct serving dynamics, where sessions are long-lived with large KV caches, and agents spend most of their time paused and idle between actions [4].

This idle period breaks the conventional assumption that cached sessions are always actively decoding. Because resume timing is unpredictable, the idle KV must either occupy scarce HBM the whole time or be offloaded to slower memory and fetched back on resume — trading HBM capacity against resume latency. As a result, the KV cache divides into two populations with opposing access patterns. A small *hot set*, the KV of the currently-decoding batch, is read on every step. Because per-request throughput declines as the batch grows [2], serving systems cap the active batch to keep this set resident in HBM. The second population is a large *cold pool*, the KV of paused sessions, which is read only on resume. Scaling with agent concurrency, this pool quickly overflows HBM, forcing systems to provision extra hardware or suffer high eviction/recomputation penalties on resume [5].

High bandwidth flash (HBF) is an emerging memory that applies vertical die-stacking and parallel-interface design of HBM to NAND [6], [11]. Its core structure is split into many independently addressable sub-arrays, which are read in parallel. Together with short intra-stack data paths, die stacking enables high bandwidth. A single stack achieves read bandwidth of about 1,638 GB/s, comparable to that of an HBM4 stack [12]. Its capacity, however, is far larger with a single stack holding roughly 512 GB against the ~ 48 –64 GB of HBM4 [12]. However, HBF inherits two limitations of flash technology. First, write endurance is limited. Each cell tolerates a limited number of program/erase cycles. Therefore, the total writable budget per stack is the tier capacity C times the rated cycle count N_{PE} . Dividing this budget by the effective write rate, which is the application write rate R_W times the write amplification factor WAF, gives the expected service lifetime:

$$T_{\text{life}} = \frac{C N_{PE}}{R_W \cdot \text{WAF}} \quad (1)$$

Because T_{life} scales inversely with R_W , the limited endurance is only tolerable when writes are infrequent, which favors infrequently-updated data. Second, reads are page-granular (tens of KB) [8], so a sub-page request still fetches and pays energy for an entire page; small or random reads therefore

waste effective bandwidth and inflate per-bit read energy [7], [13]. Prior works address flash limitations through distinct strategies. [8] restricts storage to read-only state such as model weights and pre-computed KV, [6] admits generated KV only when output-length predictions favor high read-to-write ratios, and [14] offloads all KV data using custom prefetching/buffering, though omitting read energy costs.

III. DESIGN: HOT-COLD TIERING OF HBM AND HBF

We hierarchically organize HBF and HBM, mapping active decoding KV caches to the bandwidth-dense, energy-efficient HBM, and the idle pool to capacity-dense HBF. Active KV states are written to HBF only on eviction, while a resumed session is prefetched to HBM before execution resumes.

Architecture and data paths: Fig. 2 shows the organization, which adopts the co-packaged HBM+HBF substrate proposed by [8], [14], [15]. The HBM base die fuses the two tiers with an address router and staging buffers that double-buffer inter-tier traffic, prefetching cold blocks on resume and absorbing evictions so HBF’s NAND latency hides behind the D2D transfer. While the GPU addresses HBM directly through this base die, the HBF is accessible exclusively via the D2D link. This configuration yields four distinct data paths. ① On the decode path, model weights and the active batch’s KV stream from HBM into the GPU. ② When a paused session resumes, its KV is fetched from HBF back into HBM over the D2D link. ③ Freshly generated KV is written back into HBM. ④ Under capacity pressure, the coldest idle KV is evicted from HBM to HBF over that same link. Paths ① and ③ carry the hot, per-step traffic on the GPU–HBM bus. In contrast, paths ② and ④ handle the cold-pool traffic on the HBM–HBF D2D link. The GPU therefore reads the hot set from HBM while the base-die buffers move HBF blocks, so eviction and resume overlap with decode.

Memory hierarchy and write policy: HBF can be used for the KV cache in several ways. However, it has higher latency, lower endurance, and greater read energy than HBM. *All-KV-to-flash* methods [6], [14] eventually commit every KV cache to HBF, bounding endurance to the full write stream and putting HBF on the per-step write path. We instead *write-on-evict*: a KV block is written to HBF only when it is evicted from HBM, therefore a block reaches HBF only when demoted from HBM. Because KV is immutable, a block re-admitted to HBM and evicted again does not need rewriting. The HBF copy remains valid, so *write-on-evict* writes each block to HBF at most once. This policy reduces HBF writes by confining HBF traffic strictly to eviction and resume. HBM retains model weights, active KV, and recent idle KV, while HBF acts as an overflow tier for demoted idle KV.

Eviction, resume and decode: When HBM fills up, the least-recently used (LRU) KV block is evicted, though any caching policy separating hot and cold sets suffices. Active blocks are re-read every step, therefore recency naturally keeps the active set in HBM and demotes the KV states of paused sessions to HBF. We deliberately avoid hard-pinning the KV blocks of active sessions to HBM. Pinning may deadlock when the active set alone exceeds HBM, whereas recency gracefully degrades by spilling blocks under extreme pressure rather than

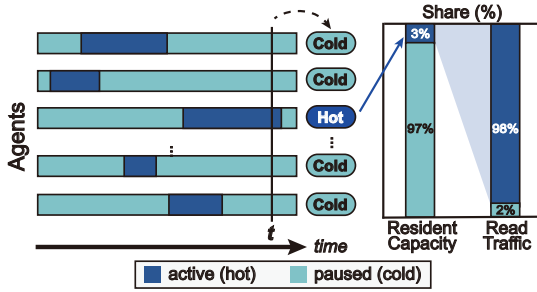


Fig. 3. Agentic KV access is highly bimodal. Left: an illustrative timeline where each agent alternates between active and paused. Right: measured shares at $N_a = 48$, where the hot set occupies 3% of resident capacity yet drives 98% of read traffic.

failing. On session resume, HBF-resident KV is fetched back into HBM for prefill. Sub-array parallelism amortizes flash read latency across the fetch, and it overlaps with the CPU-side processing (e.g., tokenization) that precedes each turn [16], keeping HBF off the critical path. Following resumption and prefill, the agent enters the decoding stage, which dominates total inference time [10]. Decode phase uses deterministic KV access patterns to prefetch required blocks one step ahead, mitigating flash-induced stalls.

IV. EVALUATION

Methodology: We evaluate trajectories collected with mini-swe-agent on SWE-bench Verified [1], a benchmark of real software engineering tasks. In our evaluation, we use Qwen3-Coder-30B-A3B [17], a Mixture-of-Experts (MoE) model with 56.8 GiB of BF16 weights and 96 KiB of KV per token. We simulate this workload in an in-house, trace-driven simulator with analytical latency and power models. As depicted in Fig. 2, the modeled node is a B200 GPU [18] with 192 GiB of HBM3e at 8 TB/s and 2.25 PFLOP/s of compute, augmented with an HBF tier alongside its HBM. The HBF tier is 3 TiB SLC with bandwidth matching HBM, rated at 100,000 program/erase cycles [19] with 25 μ s read access latency. As KV is immutable and appended in erase-block-aligned units, the flash avoids read-modify-write and garbage-collection rewrites. This keeps the WAF near unity (≈ 1.02) [6], [20], compared to 2 to 4 for a general-purpose SSD [21], leaving little room for over-provisioning to reduce it further.

To model the realistic tool execution and user idle latency, we draw each idle gap from a heavy-tailed distribution with a median of about 8 seconds and a mean of about 23 seconds, capturing tool calls that are mostly short but long-tailed [3] and user responses that add tens of seconds [22]. Due to scarce public agent traces, we scaled the workload to thousands of concurrent sessions to fully utilize HBF capacity by replaying measured trajectories with independent arrival phases and idle gaps. We track each 1.5 MiB KV block as resident in HBM, HBF, or both, and apply write-back with LRU eviction. This granularity amortizes HBF's page-granular reads, whereas smaller blocks would waste sub-page reads and larger ones would coarsen eviction. To eliminate routing-induced latency variances inherent to MoE architectures, the entire complement of expert weights is retained within HBM. A batching scheduler [2] keeps N_a sessions decoding at all times, where N_a is the number of concurrently decoding

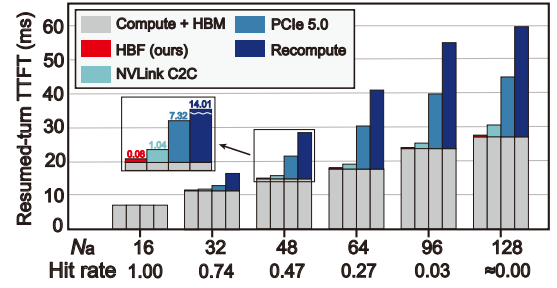


Fig. 4. Resumed-turn time-to-first-token (TTFT) by cold tier across N_a . The grey base is the latency for prefill computation with HBM resident KV after fetching. The colored top is the extra latency to restore the evicted blocks from each tier on resume. The number below N_a is the resume hit rate.

(active) agent sessions, and admits a new one whenever a pause occurs. We sweep N_a from 8 to 128, the range over which the hot set fits in HBM and the TBT SLO remains attainable. Within each session, an agent's trajectory grows to a peak context of about 16.7k tokens, or 1.53 GiB of KV. Across the concurrent sessions in steady state, where agents span all trajectory stages, a resident agent holds 0.82 GiB on average. We report this footprint, the resume hit rate (the fraction of a resuming session's KV resident in HBM), the cold-fetch and write volume over the saturated plateau of the replay, excluding its warm-up and drain [23].

We model read power as the sum of each memory tier's read rate multiplied by its per-bit energy. Since HBF read energy is an unstandardized value, we treat it as a parameter and sweep it from 8 to 30 pJ/bit [7] against HBM's ≈ 3.5 pJ/bit [24]. We compare four baselines: *All-KV-to-flash* method, which serves all KV from flash, CPU offload over PCIe 5.0 at 64 GB/s, or NVLink-C2C at 450 GB/s [25] and recomputation.

Bimodal access: Fig. 3 confirms this bimodality on our workload. At $N_a = 48$ the hot set is about 96 GiB, fitting in HBM with headroom, yet it draws about 98% of read traffic while occupying 3% of KV capacity.

Capacity and latency: An HBM-only device saturates at 165 sessions for the simulated trace, whereas adding a 3 TiB HBF tier expands capacity to 3,900 sessions. However, capacity alone is not the deciding factor, since host CPU memory offers ample capacity as well. A critical requirement is that the memory must be fast enough to restore a session within a turn's latency budget. Fig. 4 evaluates the resumed-turn TTFT by varying the cold-pool interconnect within our hot-cold framework. As concurrency N_a rises, the resume hit rate drops toward zero, increasing block misses and causing restore latencies to scale across the tiers. At $N_a = 48$, HBF adds 0.084 ms of overhead while NVLink-C2C adds 1.0 ms and PCIe adds 7 ms. Recomputation consumes GPU compute to re-prefill evicted blocks, adding a 14 ms overhead at $N_a = 48$ and over doubling turn cost at $N_a = 128$. Conversely, HBF fetches operate as pure IO off the compute path. HBF offers capacity to hold a large cold pool at a negligible resume cost.

Since resuming a request restores its KV into HBM, the subsequent decode phase always reads KV from HBM, leaving TBT independent of the cold tier. What matters for decode is tiering itself. Without it, the hot set may reside outside HBM and must be retrieved from the backing tier at every decoding step, failing to meet strict per-user TBT SLOs. As

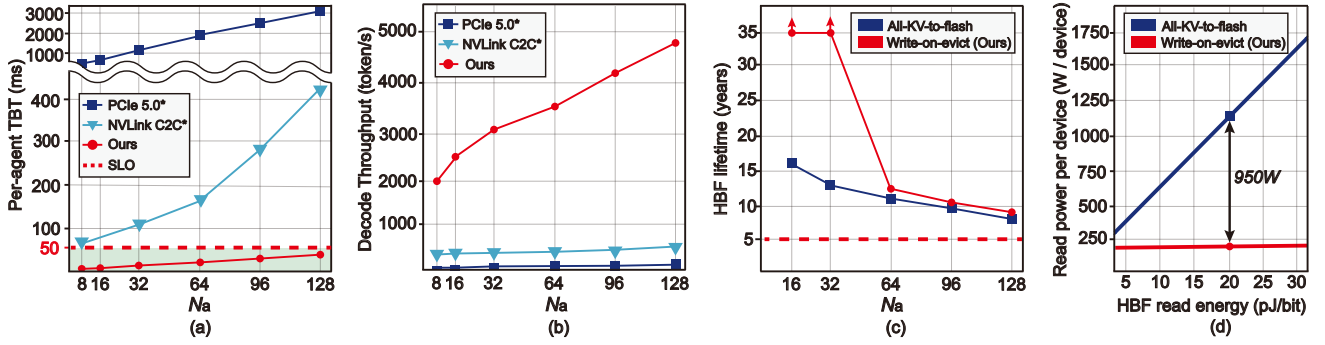


Fig. 5. (a) Per-agent TBT versus N_a . Asterisks(*) mark configurations without hot-cold tiering, where decode reads the hot set from the backing tier instead of HBM. (b) Aggregate decode throughput for the same comparison. (c) HBF lifetime versus N_a . (d) Read power versus HBF read energy at $N_a = 48$. At 20 pJ/bit, the gap is about 950 W per device.

shown in Fig. 5(a), when accessing the backing tier over PCIe or NVLink, neither interconnect can sustain the bandwidth required by the hot set, leading to SLO violations. In contrast, our design with tiering reports 14 ms per token at $N_a = 48$ (72 tok/s) and 27 ms at $N_a = 128$ (37 tok/s) per agent, both safely within the 50 ms SLO. Hence, keeping the frequently read active KV on-package is crucial to meet per-step latency requirements. Our design therefore bounds N_a by how much KV HBM can hold. Within this bound, N_a trades per-agent TBT against aggregate throughput, as Fig. 5(b) shows throughput rising as N_a scales. The HBF tier holds the large idle pool cheaply, which suits premium latency-sensitive serving rather than throughput-maximizing batch inference.

Endurance: Endurance is the main concern with HBF as a KV storage [6], making lifetime a key factor. Fig. 5(c) plots HBF lifetime against N_a for both write policies. Even *all-KV-to-flash*, which writes every produced KV block to HBF, clears the 5-year warranty [6] across the load, falling from about 11 years at $N_a = 48$ to about 8 years at $N_a = 128$ as the write rate rises with throughput. *Write-on-evict*, which writes each KV block to HBF at most once on eviction and skips blocks that never leave HBM, sits well above it. At N_a below 32, nothing is evicted and lifetime is effectively unbounded, and at $N_a = 48$ it still reaches about 20 years. The two curves converge at high N_a , where the working set outgrows HBM and *write-on-evict* must evict nearly as often as *all-KV-to-flash*. Endurance is therefore not a barrier to placing the cold pool in HBF but an axis our design improves.

Read energy: Serving all KV from flash [6], [14] pays HBF's read energy on every actively read KV, whereas our split leaves HBF carrying only the cold pool, read once per resume. Across the load, the read rate rises from 6 GB/s at $N_a = 16$ to 38 GB/s at $N_a = 128$. Fig. 5(d) highlights the power difference between *all-KV-to-flash* and *write-on-evict* policies. Assuming an HBF energy of 20 pJ/bit based on the power budget in [8], a *write-on-evict* policy saves about 950 W per device, or 7.6 kW across an 8-GPU node. The difference scales with the energy gap between HBM and HBF, from about 2.1 to 12.2 kW per node across the swept read-energy range. This result shows that sourcing hot KV from HBF rather than HBM is impractical from a power consumption perspective.

V. CONCLUSION

Prior HBF designs either place read-only KV in HBF [8] or place all KV in HBF [6], [14]. We instead match the memory

hierarchy to the bimodal access pattern of agentic LLM KV. The hot set read every step stays in HBM while only the cold idle pool moves to capacity-optimized HBF. This supports an order of magnitude more concurrent sessions per GPU and cuts memory-read power by about 7.6 kW per 8-GPU node versus *all-KV-to-flash*, with decoding running entirely on HBM. HBF thus serves as an efficient cold tier in a hot-cold KV hierarchy, not a replacement for HBM.

REFERENCES

- [1] SWE-bench Team, "SWE-bench-Verified and mini-swe-agent," 2025, <https://www.swebench.com/verified.html>.
- [2] A. Agrawal *et al.*, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *OSDI*, 2024.
- [3] H. Li *et al.*, "Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live," *arXiv*, 2026.
- [4] Z. Bian *et al.*, "Tokencake: A kv-cache-centric serving framework for llm-based multi-agent applications," *arXiv*, 2026.
- [5] S. Qiu *et al.*, "Tutti: Making ssd-backed kv cache practical for long-context llm serving," *arXiv*, 2026.
- [6] K. Kyung *et al.*, "High-bandwidth flash for KV caches: Endurance and performance implications," *IEEE CAL*, 2026.
- [7] P. Hsu *et al.*, "Haven: High-bandwidth flash augmented vector engine for large-scale approximate nearest-neighbor search acceleration," 2026.
- [8] M. Ha *et al.*, "H3: Hybrid architecture using high bandwidth memory and high bandwidth flash for cost-efficient llm inference," *IEEE CAL*, 2026.
- [9] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *SOSP*, 2023.
- [10] K. Jiin *et al.*, "The cost of dynamic reasoning: Demystifying ai agents and test-time scaling from an ai infrastructure perspective," in *HPCA*, 2026.
- [11] D. Son *et al.*, "Exploring high-bandwidth flash for modern llm inference: Opportunities and challenges," *IEEE Computer Architecture Letters*, vol. 25, no. 2, pp. 251–254, 2026.
- [12] X. Ma and D. Patterson, "Challenges and research directions for large language model inference hardware," *Computer*, vol. 59, 2026.
- [13] J. H. Ju *et al.*, "TileLens: Efficiently using large-granularity memory systems with transparent two-dimensional memory layout," *arXiv preprint arXiv:2607.04031*, 2026.
- [14] J. Park *et al.*, "HBM-HBF-Centric Memory Pooling Architecture With Custom Base Die for Terabyte-Scale LLM Inference," *IEEE CAL*, 2026.
- [15] X. Wang *et al.*, "FlashAccel: Leveraging high-bandwidth flash for high-throughput llm inference," *arXiv preprint arXiv:2607.10186*, 2026.
- [16] E. Chung *et al.*, "Characterizing CPU-induced slowdowns in multi-GPU LLM inference," *arXiv*, 2026.
- [17] A. Yang *et al.*, "Qwen3 technical report," *arXiv*, 2025.
- [18] Nvidia, "Nvidia dgx b200," 2025, <https://resources.nvidia.com/en-us-dgx-systems/dgx-b200-datasheet?ncid=no-ncid>.
- [19] Micron, "P/e cycle endurance for slc nand," 2026, <https://www.micron.com/products/storage/nand-flash>.
- [20] I. Jeong *et al.*, "HiFC: High-efficiency flash-based KV cache swapping for scaling LLM inference," in *NeurIPS*, 2026.
- [21] Micron, "Write amplification factor for general ssds," 2025. [Online]. Available: <https://www.crucial.com/support/articles-faq-ssd/why-does-ssd-seem-to-be-wearing-prematurely>
- [22] L. Yu *et al.*, "Stateful large language model serving with Pensieve," in *EuroSys*, 2025.
- [23] Q. Chen *et al.*, "Concur: High-throughput agentic batch inference of llm via congestion-based concurrency control," 2026.
- [24] M. Adiletta *et al.*, "Rpu – a reasoning processing unit," in *HPCA*, 2026.
- [25] Nvidia, "Nvidia grace hopper superchip architecture in-depth," 2022, <https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/>.